

Sprawozdanie

Project Gate

Autorzy:

Marcin Przybyłek

Justyna Piotrowska

Agnieszka Zięcina

Marek Brzyski

Pierwotny zamysł

Naszą ideą jest stworzenie gry, którą można bez problemu rozgrywać wiele razy. Celujemy w graczy lubiących akcję i wyzwania, a także będących w stanie poradzić sobie w sytuacji bycia ciągle przytłoczonym przez przeciwnika.

Gracz będzie zarządzał małym elitarnym oddziałem, mającym na celu zabezpieczenie portali, z których dużymi falami wychodzą przeciwnicy. Do tego celu gracz będzie miał możliwość wykorzystania różnych rodzajów broni palnej, zdolności magicznych swoje oraz swojego oddziału i możliwość wydawania rozkazów AI.

Docelową platformą naszej gry będzie PC VR.

Pierwotny plan pracy

	23.03	30.03	13.04	04.05	18.05	01.06
Marcin Przybyłek	Konfiguracja aplikacji		Podstawowa kontrola gracza	Implementacja broni i korzystania z nich	Wprowadzenie menu taktycznego i UI	Stworzenie specjalnych umiejętności gracza
Justyna Piotrowska		Pierwsza wersja generowania terenu – sam kształt	Mapa kolorów terenu na podstawie szumu, zaprojektowanie mapy do fazy planowania	Obsługa tekstur, shaderów i oświetlenia, dodanie obiektów do terenu – drzewa, skały, struktury	Dodanie różnych typów terenu do generatora – miasta, lasy, pustkowia	Wprowadzenie eliminowania niewidocznych powierzchni
Agnieszka Zięcina	Przygotowanie wstępnych szkiców przeciwników		AI przeciwników (5 przeciwników)	AI przeciwników (5 przeciwników)	Animacje szkieletowe	AI NPC
Marek Brzyski	Rozpisanie umiejętności i typów przeciwników		AI przeciwników (5 przeciwników)	AI przeciwników (5 przeciwników)	Projektowanie działania pętli gry	

Czego nie udało się zrealizować

Z powodu naszego braku doświadczenia w pracy zespołowej w silniku Unity oraz braku doświadczenia w tworzeniu gier VR mieliśmy spore opóźnienia w stosunku do naszych zamierzonych celów. W wyniku tego nie udało się zrealizować:

- Sojuszników NPC
- Umiejętności gracza
- Generowania struktur
- Systemu progresji

Implementacja

Przeciwnicy - Agnieszka Zięcina

Klasa bazowa- OpponentBase

OpponentBase jest abstrakcyjną klasą bazową, która definiuje wspólne cechy i zachowania przeciwników w grze. Wykorzystanie jej w dziedziczeniu pozwoliło na ujednolicenie logiki poruszania się, atakowania, obrażeń czy paska zdrowia.

Najważniejsze statystyki:

maxHealth – określa początkową i maksymalną wartość zdrowia dla danego typu przeciwnika

currentHealth - określa aktualną ilość zdrowia przeciwnika

defense - określa wartość defensywy przeciwnika

baseAttack – określa wartość ataku, co przekłada się bezpośrednio na siłę ataków

enemyValue - to wartość liczbowa określająca względny poziom trudności danego przeciwnika. Jest wykorzystywana w systemie generowania przeciwników (spawnowania) do zbalansowania całkowitej siły wrogów pojawiających się na polu bitwy

speed – określa szybkość poruszania się przeciwników

Atakowanie:

Podstawowa funkcja Attack(). jeśli gracz jest w zasięgu, zadaje obrażenia co odstęp czasowy określony przez zmienną damageInterval.

Pasek zdrowia:

Pasek zdrowia generowany jest nad przeciwnikiem, aktualizuje się dynamicznie i jego kolor przechodzi od zielonego do czerwonego w zależności od poziomu HP.

Śmierć:

W momencie osiągnięcia 0 HP, przeciwnik odtwarza efekt cząsteczkowy imitujący krew, niszczy pasek życia i usuwa obiekt z gry po 2 sekundach.

NavMesh:

Podczas inicjalizacji obiekt próbuje automatycznie dopasować się do NavMesha, co eliminuje problemy z rozmieszczeniem jednostek.

Opis Przeciwników

Skrzydlaty demon (FlyingDemon)

Latający przeciwnik, który przy ataku pikuje w dół, a następnie wraca w powietrze. Kiedy

nie atakuje, utrzymuje pozycję nad ziemią, stale śledząc gracza.

Atak został zaimplementowany jako coroutine DiveAttack(), podczas którego NavMeshAgent jest tymczasowo wyłączany, a obiekt manualnie porusza się w stronę gracza, po czym wraca do pozycji startowej. Dodatkowo UpdateModelPosition() dba o płynne unoszenie modelu w trakcie lotu.

Model: Asset z Unity AssetStore – Andryuha198:

<https://assetstore.unity.com/packages/3d/characters/animals/insects/flying-carnivorous-beetle-95575>

Animacje: Przypisane do modelu, używane z Animatora.

Wampir (Vampire)

Klasyczny przeciwnik z umiejętnością regeneracji. Przy każdym ataku ma szansę przywrócić sobie niewielką ilość HP.

Zmienna noHealAttack śledzi liczbę ataków bez leczenia, co pozwala warunkowo odpalić mocniejszą formę regeneracji po trzecim ataku.

Model: SketchFab – Pups1k, licencja CC Attribution.

<https://sketchfab.com/3d-models/vampire-character-model-3d4fd35d383c4b548d4c08a0918a7a8a>

Animacje: Adobe Mixamo.

Nekomanta (Nekomancer)

Przeciwnik sam nie angażuje się w walkę, zamiast tego przyzywa do trzech nieumarłych, którzy walczą za niego.

Funkcja SummonMinion() losowo wybiera pozycję w promieniu summonRadius, upewniając się, że nie koliduje z innymi obiektami. Lista summonedMinions pozwala kontrolować aktywne jednostki i czyści referencje do zniszczonych obiektów.

Model: Autorski model – Izabela Piotrowska.

Animacje: Własnoręcznie wykonane w Unity z użyciem Animatora i narzędzia Animation.

Kamienny golem (Golem)

Sporej wielkości przeciwnik, o dużych hitboxach. Ataki są silne, ale jedynie blisko-dystansowe.

Model: Autorski model – Izabela Piotrowska.

Animacje: Własnoręcznie wykonane w Unity z użyciem Animatora i narzędzia Animation.

Leśny duszek (Forest Spirit)

Jednostka wsparcia, która leczy innych przeciwników co 10 sekund, ale nie potrafi leczyć samego siebie. W przypadku pozostania samotnym na polu bitwy przełącza się w tryb agresywny, w którym zadaje duże obrażenia.

Dwa tryby działania (leczenie/agresja) są kontrolowane przez zmienną `isAggressive`, która aktualizuje się w `CheckIfAlone()` na podstawie obecności sojuszników wykrytych przez `Physics.OverlapSphere()`. Leczenie odbywa się za pomocą przypisanej warstwy `allyLayer`

Model: Autorski model – Izabela Piotrowska.

Animacje: Własnoręcznie wykonane w Unity z użyciem Animatora i narzędzia Animation.

Wilkołak

Wilkołak jest szybkim przeciwnikiem, który podbiega do gracza, a następnie krąży wokół niego, starając się wykonać atak od tyłu.

Ruch okrężny wokół gracza realizowany jest przez zmianę pozycji `orbitTarget`, a atak od tyłu aktywowany jest tylko wtedy, gdy przeciwnik znajduje się w określonym kącie za plecami gracza (sprawdzanym przez `Vector3.Angle`). Zmienna `isCircling` decyduje, czy wilkołak orbituje, czy goni gracza bezpośrednio.

Model : SketchFab – silvercro101, licencja CC Attribution

<https://sketchfab.com/3d-models/werewolf-2dd83cb880f047d3bb2085f13e1efc08>

Animacje: Adobe Mixamo

Jadowity wąż

Niewielki, ale nietrudny przeciwnik.

Po zadaniu obrażeń w funkcji `DoAttack()`, wąż może rozpocząć coroutine `PoisonCoroutine()`, która przez 5 sekund co sekundę zadaje dodatkowe obrażenia. Zmienna `isPoisoned` zapobiega nakładaniu efektu wiele razy jednocześnie.

Model : SketchFab – ChristineDesigns, licencja CC Attribution.
<https://sketchfab.com/3d-models/snake-75578506d748405fb0fdc6426aa103d7>

Animacje : Własnoręczne, wykonane w Unity z użyciem Animatora i narzędzia Animation.

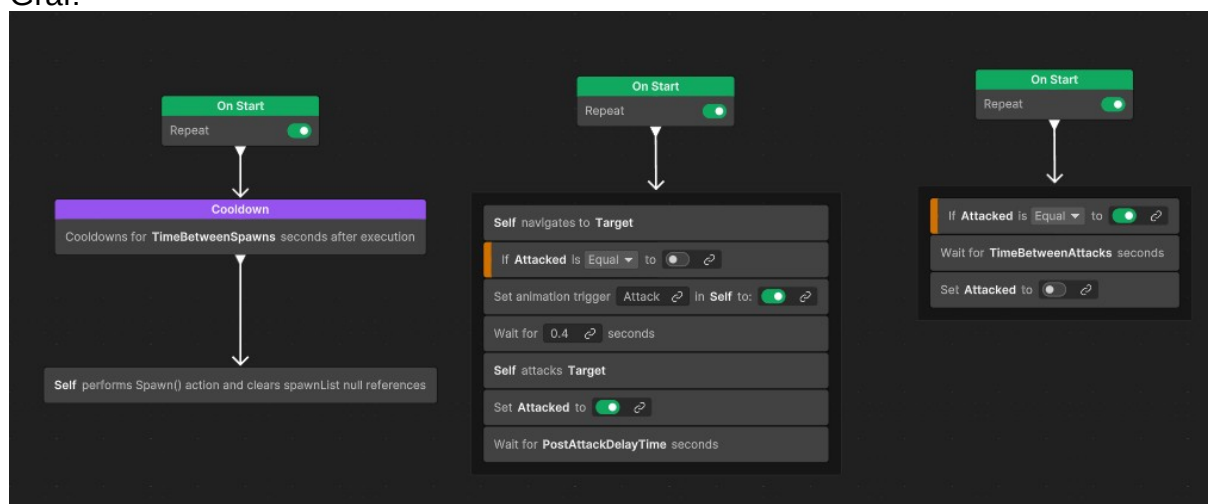
Przeciwnicy z użyciem grafów behawioralnych - Marek Brzyski

Część przeciwników została zrobiona z użyciem grafów behawioralnych. Ich klasy są rozwinięciem bazowej `OpponentBase` i zawierają wszystkie potrzebne funkcje, jednak zachowanie i wywoływanie funkcji jest sterowane osobnym komponentem `Behavior Agent` w którym znajduje się graf behawioralny. Grafy te umożliwiają szybką i łatwą zmianę zachowania potworów jak trzeba oraz pozwalają tworzyć złożone ale jednocześnie czytelne wzorce zachowań. Możliwe jest tworzenie własnych bloków akcji do struktury grafu ale też używanie wbudowanych, dzięki czemu proces jest szybki i przyjemny, a można osiągnąć nawet lepsze efekty niż przy standardowym pisaniu zachowania w kodzie przeciwnika.

Lich

Przeciwnik który przywołuje innego przeciwnika, Minion, do kilku istniejących w jednym momencie, ilość określona parametrem `maxSpawns`. Posiada on wewnętrzną listę przeciwników śledzącą ile minionów aktualnie jest. Przy pojawieniu się przywołuje tylu przeciwników ile określa `onStartSpawns`, a następnie co 15 sekund kolejnego, równoległe do pozostałych akcji. Służy do tego metoda `Spawn()`. Ponadto przeciwnik ten strzela magicznymi strzałami do gracza. Pociski są zaimplementowane w prefabie `LichProjectile` z ich działaniem w klasie `.cs` o tej samej nazwie.

Graf:



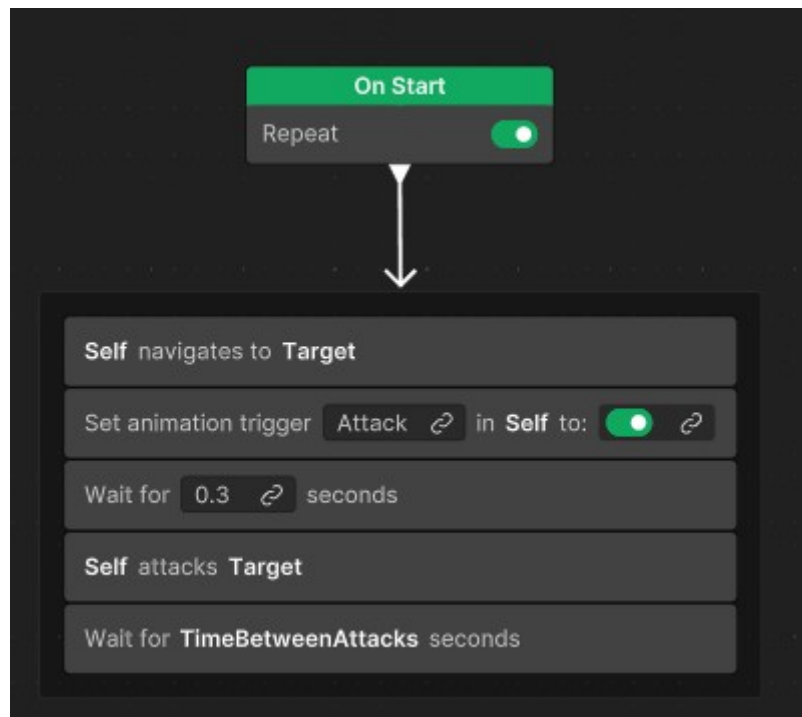
Zastosowanie kilku `On Start` zamiast bloku `Run In Parallel` wynika z faktu, iż tamten blok czeka aż wszystkie równoległe ścieżki się zakończą zanim zostanie uruchomiony ponownie, co w przypadku nawigowania do gracza zatrzymuje procesy poboczne, jak przywoływanie przeciwników.

Asset: <https://kaylousberg.itch.io/kaykit-skeletons>

Minion

Prosty przeciwnik walczący w ręcz. Jest podstawowym potworem z ilością punktów 1. Służy ponadto jako przeciwnik który jest przywoływany przez Licza i Nekomantę.

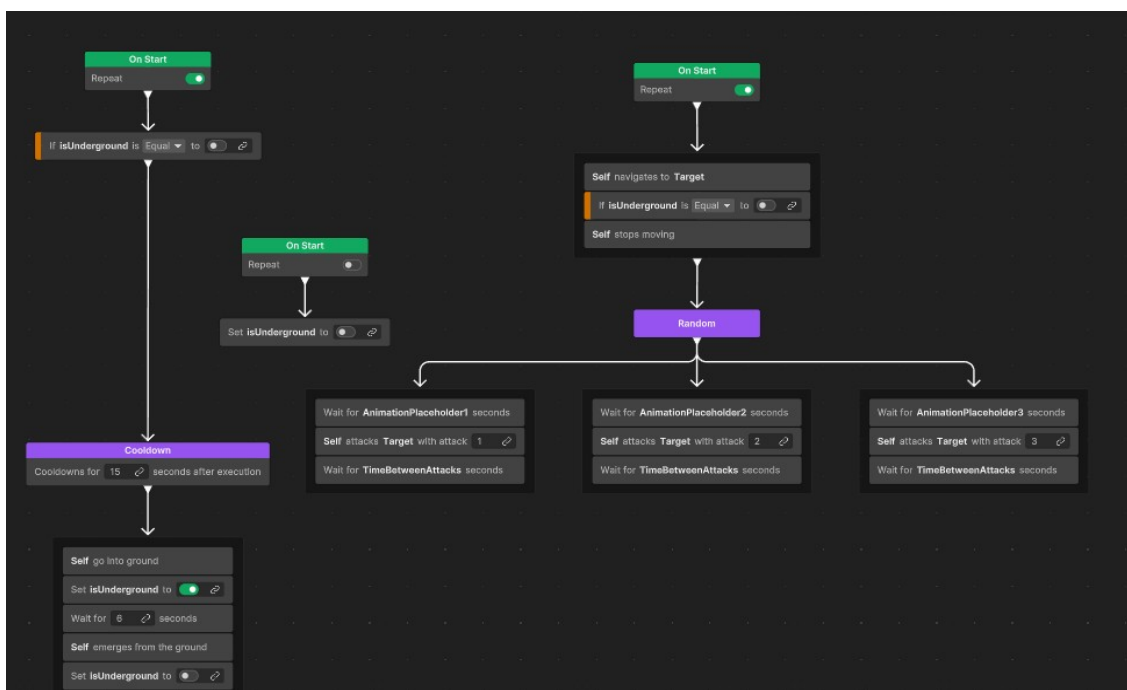
Graf:



Asset: <https://kaylousberg.itch.io/kaykit-skeletons>

Shadow Panthershark

Przeciwnik bliskozasięgow, który jest szybki i co jakiś czas wnika w ziemię na 5 sekund, stając się nie do trafienia oraz tworząc w miejscu w którym jest na powierzchni fioletowo czarne koło. Służą do tego metody: `HideObjectUnderground()`, `CreateGroundProjection(Vector3 groundPosition)`, `RestoreObject()`. Na czas „ukrycia” komponent Collider jest ustawiany na `isTrigger`, żeby przeciwnik mógł się poruszać, ale nie dało się go trafić. Przy całkowitym wyłączeniu Collidera Nav Mesh Agent przestaje działać. Posiada 3 rodzaje ataku z czego wybierany jest losowy. Niestety przez brak modelu przeciwnik nie został wprowadzony, co może się zmienić w przyszłości.



Portal – Marek Brzyski

Portal służący do przyzywania potworów, jego działanie znajduje się w skrypcie `MonsterSpawner.cs`. Znajduje się w losowym miejscu na mapie i trzeba się do niego zbliżyć, aby włączyć fazę obrony, w której trzeba przetrwać w pobliżu portalu przez określony czas.

Obrona działa na zasadzie punktów, co sekundę kiedy gracz znajduje się w strefie dodawane jest określone `defenseUpValue`, a jak nie ma go w niej odejmowane określone `defenseDownValue` przy czym nie niżej niż 0. Tak się dzieje aż zostanie osiągnięte `defenseMaxTime`. Następnie przeciwnicy przestają się pojawiać i po pewnym czasie gracz jest przenoszony. Wszystkie te fazy są sterowane odpowiednimi flagami w `Update()`.

Samo przywoływanie przeciwników również działa na zasadzie punktów. Portal ma przydzielony `enemyPointLimit`, a każdy przeciwnik posiada swoją ilość punktów potrzebną do przyzywania. Przeciwnicy o punktach ≥ 5 są traktowani jako zaawansowani. Ilość punktów potwora jest odejmowana od puli punktów portalu, a przy śmierci przywracana.

Portal ma zakres 1-100 i wartość graniczną 80. Przy generowaniu przeciwników losowana jest liczba z zakresu i jak jest większa bądź równa wartości granicznej przyzywany jest przeciwnik zaawansowany, jak niższa to podstawowy. W przypadku niewystarczającej liczby punktów w portalu wybierany jest taki przeciwnik który może być przyzwany aby punkty nie były niższe niż 0.

Aby zapewnić różnorodność przy przyzywaniu przeciwnika modyfikowane są zakresy liczb. W przypadku podstawowego przeciwnika dolna granica przesuwana jest w górę o określoną ilość. Analogicznie przeciwnik zaawansowany przesuwą górną granicę.

Sterowane jest to w funkcji `SpawnMonsters()`. Poza tym jest `InitialSpawnMonsters()` które wyzwalane jest na początku i przywołuje potwory aż do połowy `enemyPointLimit`. Są również funkcje generujące transparentne sfery zieloną i czerwoną, obrazujące odpowiednio zakres do którego trzeba wejść aby uruchomić fazę obrony oraz zakres w którym trzeba pozostać aby licznik obrony wzrastał.

Asset: <https://assetstore.unity.com/packages/3d/environments/fantasy/runesandportals-85098>

Bronie palne - Marcin Przybyłek

Bronie w grze składają się z 3 interaktywnych elementów: samej broni, zamka i magazynka. Magazynek jest możliwy do włożenia tylko do odpowiedniego typu broni i przechowuje on informacje o ilości pocisków znajdujących się w środku.

W celu użycia broni należy włożyć do niej magazzynek posiadający amunicję i złapać za zamek. W tym momencie jeden z pocisków z magazynka zostanie przechowany w komorze broni. W tym momencie można użyć broni poprzez pociągnięcie za spust.

Skrypt broni wykorzystuje tylko pocisk znajdujący się w komorze i przy strzale ładuje kolejny pocisk z magazynka. Skrypt został napisany w sposób uniwersalny, dzięki czemu przy jego użyciu można stworzyć prefaby dowolnych rodzaj broni palnych. Umożliwia on skonfigurowanie trybu ognia danej broni (automatyczna, pół-automatyczna), prędkość strzału, obrażenia itp.

Wykorzystane assety:

<https://assetstore.unity.com/packages/3d/props/weapons/low-poly-ar-weapon-pack-1-276916>

<https://assetstore.unity.com/packages/vfx/particles/war-fx-5669>

Kabury – Marcin Przybyłek

Postać gracza jest wyposażona w kaburę pozwalającą przetrzymać broń przy postaci gracza. Podąża ona za graczem i obraca się wraz z nim, dzięki czemu zawsze znajduje się w okolicy biodra. Po upuszczeniu broni wewnątrz sfery reprezentującej kaburę zostanie ona do niej doczepiona.

Stan gracza między scenami – Marcin Przybyłek

Stan gracza jest zapisywany w celu przeniesienia go do innej sceny. W tym celu stworzony został ScriptableObject zawierający prefaby broni i magazynków i przypisanymi im id. Dzięki temu skrypt zapisu może zapisać dane na dysku w postaci liczb. Sam zapis odbywa się do pliku na dysku w postaci pliku json, w którym zapisane są informacje o tym czy i jaką broń gracz przetrzymuje w kaburze oraz czy broń posiada magazynek i jaki. Zapisane są również informacje dotyczące pozycji broni w kaburze, dzięki czemu ułożenie broni nie zmienia się między scenami.

Generowanie terenu – Justyna Piotrowska

Generacja terenu oparta jest na szumie Perlina i składa się z dwóch skryptów:

NoiseMap

Który tworzy mapę wysokości na podstawie szumu Perlina i pozwala dostosować wartości

- *scale* – opisującą „przybliżenie” na fragment szumu co ma wpływ na to jak bardzo szybkie są zmiany terenu
- *octaves* – opisującą ile warstw szczegółów powinna mieć mapa wysokości
- *persistence* – określający jak szybko maleje amplituda kolejnych oktaw, co pozwala zmodyfikować jak duży wpływ na obecną mapę wysokości mają następne oktawy
- *lacunarity* – opisującą zmianę częstotliwości kolejnych oktaw, co pozwala ustalić jak duże szczegóły dodają kolejne warstwy

To w jaki sposób mapowane są punkty na mapie szumu do mapy wysokości jest zrobione w następujący sposób:

$$x_{perlin} = \left(\frac{x_{map}}{scale} * frequency_i + randomValue \right) * amplitude_i$$

$$y_{perlin} = \left(\frac{y_{map}}{scale} * frequency_i + randomValue \right) * amplitude_i$$

gdzie:

i – obecna oktawa

$$amplitude_i = amplitude_{(i-1)} * persistence$$

$$frequency_i = frequency_{(i-1)} * lacunarity$$

przy czym wartości dla pierwszej oktawy to:

$$amplitude = 1$$

$$frequency = 1$$

Terrain Generator

Który edytuje teren na podstawie wygenerowanej mapy wysokości a następnie wykonuje wygładzanie poprzez porównywanie wysokości sąsiadujących wysokości mapy terenu.

Wygładzanie pozwala pozbyć się artefaktów wynikających z różnicy rozdzielczości między mapą wysokości na obiektem terenu w Unity, sprawiając że wygenerowany teren wygląda lepiej.

Następnie na mapie rozmieszczane są elementy takie jak:

- Gracz
- Portal
- Drzewa
- Skały

Pierwszym etapem rozmieszczania elementów jest utworzenie siatki o określonym rozmiarze. Służy ona do przechowywania dostępnych miejsc, w których można розміścić elementy. Następnie wstawiane zostają obiekty gracza i portalu, a pozostałe wolne miejsca wypełnione są losową ilością drzew i kamieni ustaloną z podanego zakresu.

Wykorzystane assety:

<https://assetstore.unity.com/packages/3d/environments/fantasy/fristy-stylize-modular-assets-2-180352>

Wnioski

Projekt okazał się zbyt ambitny jak na nasze możliwości, przez co nie udało nam się stworzyć zadowalającego efektu. Jednym z największych przeszkód jakie napotkaliśmy w trakcie pracy nad projektem było dobra organizacja projektu w Unity, przez co nawigacja po Assetach była utrudniona. Dodatkowo przez to że nie wiedzieliśmy, które pliki Unity powinny być ignorowane przy tworzeniu commitów zdarzały nam się konflikty na plikach które nie dotyczyły bezpośrednio naszej pracy co znowu zajmowało dużo naszego czasu.

Niemniej jednak dzięki temu projektowi udało nam się zaznajomić z procesem tworzenia gier na VR w silniku Unity oraz problemach, które należy rozwiązywać w takim projekcie.

Dowiedzieliśmy się między innymi:

- Jak działają interakcje z otoczeniem w VR oraz jak wykorzystać zdarzenia z nimi związane w skryptach
- W jaki sposób działa proceduralne generowanie terenu
- Jak działać z modelami dostarczonymi przez artystę, a także tworzyć do nich animacje
- Jak tworzyć modele zachowań bazujące na grafach behawioralnych